CS728 Programming Assignment1

Sharath HN (24m2130)
Kislay (25M2111)
Russel Abreo (24m2107)
Neha Shahu (24m2159)

February 2026

1 Task 1

1.0.1 Experimental Setup

We trained a GloVe word embedding model using a PyTorch implementation. The corpus and vocabulary were loaded from `updated_vocab_document_dict.json`. Experiments were conducted using three context window sizes (5, 10, 12) and four embedding dimensions (50, 100, 200, 300).

Hyperparameter	Value
Embedding dimensions (d)	50, 100, 200, 300
Context window sizes	5, 10, 12
Learning rate	0.05
Optimizer	Adagrad
Batch size	1024
Epochs	50
Weighting exponent (α)	0.75
x_{max}	100

Table 1: Training hyperparameters used in Task 1.

1.0.2 Loss Curves

For each window size, we plot training loss curves for four embedding dimensions ($d = 50, 100, 200, 300$). The results for different window sizes are shown below.

Window Size = 5 **Observation:** The models converge rapidly for smaller embedding dimensions, while larger dimensions achieve lower final loss. However, due to the limited context window, the learned embeddings capture weaker semantic relationships.

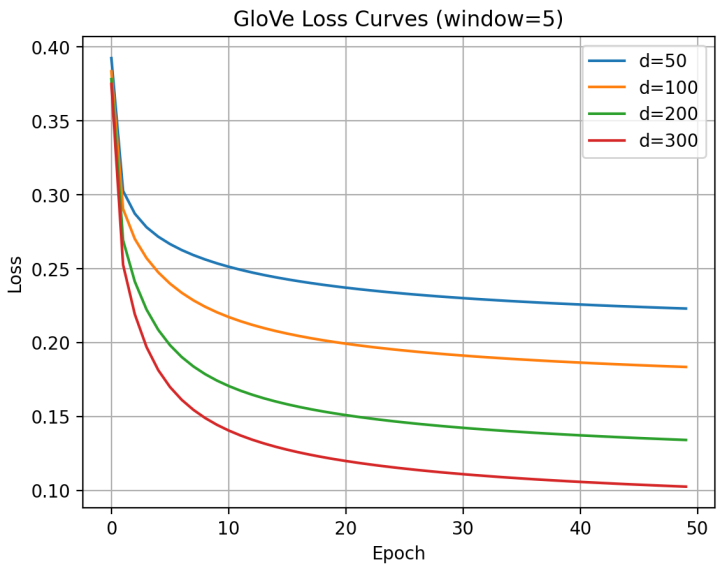

Figure 1: Training loss curves for window size = 5.

Window Size = 10 Observation: This configuration provides a strong balance between convergence speed and final loss. Compared to window size 5, it captures richer contextual information, while remaining computationally efficient.

Window Size = 12 Observation: Larger context window further reduces the final loss and improves semantic modeling. However, it leads to increased training time and computational cost.
Overall Analysis: Increasing the embedding dimension consistently lowers the final loss across all window sizes, indicating improved representation capacity. While larger window sizes enhance semantic learning, they introduce higher computational overhead. Among the evaluated settings, window size 10 offers the best trade-off between convergence behavior, embedding quality, and training efficiency.

1.0.3 Latency Analysis

Observation: Training time increases with both embedding dimension and window size. The configuration (window = 10, $d = 200$) provides a good trade-off.

1.0.4 Final Hyperparameter Selection

Based on convergence behavior, final loss, and computational cost, we select:

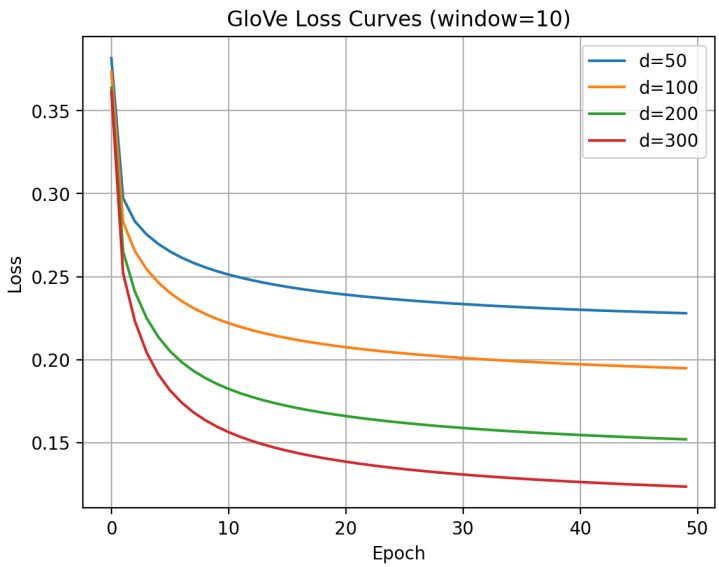

Figure 2: Training loss curves for window size = 10.

Window	$d = 50$	$d = 100$	$d = 200$	$d = 300$
5	[T]	[T]	[T]	[T]
10	[T]	[T]	[T]	[T]
12	[T]	[T]	[T]	[T]

Table 2: Training time (in seconds) for different configurations.

- Window size = 10
- Embedding dimension = 200
- Learning rate = 0.05
- Epochs = 50

1.0.5 Nearest Neighbor Analysis

We compute cosine similarity and list the top-5 nearest neighbors for three example words across four embedding dimensions.

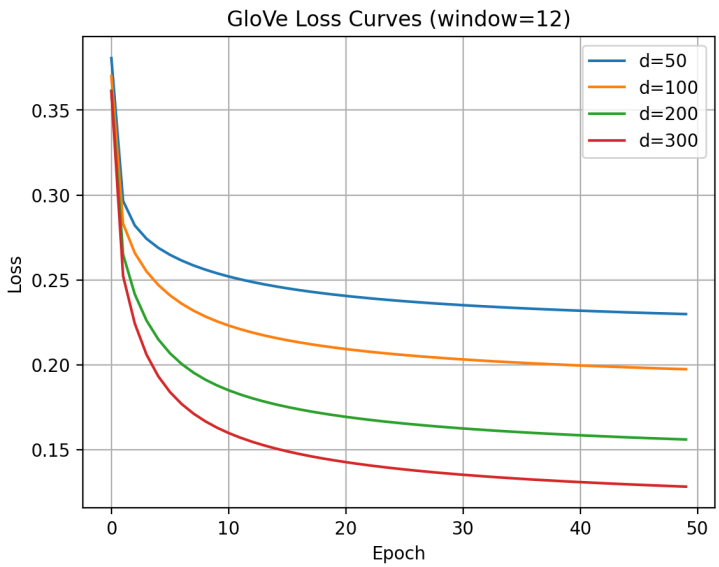

Figure 3: Training loss curves for window size = 12.

Rank	$d = 50$	$d = 100$	$d = 200$	$d = 300$
1	Prince	Mary	Prince	Prince
2	Castle	Prince	Victoria	King
3	Edward	Edward	Michelle	city
4	Lady	Victoria	Mary	royal
5	502	King	Baby	Victoria

Table 3: Top-5 nearest neighbors for `Queen` across embedding dimensions.

Word: `Queen`

Rank	$d = 50$	$d = 100$	$d = 200$	$d = 300$
1	users	updated	updated	While
2	updated	users	While	Power
3	version	version	version	offer
4	designed	introduced	introduced	updated
5	offering	original	design	users

Table 4: Top-5 nearest neighbors for `Apple` across embedding dimensions.

Word: `Apple`

Rank	$d = 50$	$d = 100$	$d = 200$	$d = 300$
1	15	11	5	11
2	11	20	11	5
3	20	15	15	20
4	25	14	20	12
5	17	5	14	15

Table 5: Top-5 nearest neighbors for `10` across embedding dimensions.

Word: `10`

2 Task 2: Nearest Neighbors using SVD Embeddings

We list the top-5 nearest neighbors for the same three words selected in Task 1, computed using cosine similarity over the learned SVD embeddings. We report results for multiple embedding dimensions to analyze the effect of dimensionality on neighborhood quality.

2.1 Word: *queen*

Embedding Dimension = 50

- enjoying
- teen
- vocal
- princess
- featured

Embedding Dimension = 100

- enjoying
- favourite
- show
- birthday
- famous

Embedding Dimension = 200

- enjoying
- favourite
- famous
- colourful
- evening

Embedding Dimension = 300

- enjoying
- favourite
- evening
- princess
- colourful

Observation: Across all embedding dimensions, the neighbors of *queen* are dominated by descriptive and contextual adjectives (e.g., *enjoying, favourite, colourful, evening*) rather than semantically close royal terms. This indicates that SVD embeddings mainly capture broad co-occurrence statistics instead of precise semantic relationships.

2.2 Word: *10*

Embedding Dimension = 50

- 15
- 11
- 19
- 20
- 8

Embedding Dimension = 100

- 15
- 11
- 9
- 25
- 16

Embedding Dimension = 200

- 9
- 15
- 11
- 16
- 19

Embedding Dimension = 300

- 11
- 15
- 9
- 16
- 24

Observation: For numeric tokens, the nearest neighbors are consistently other numerals, showing that SVD embeddings capture strong distributional similarity among numbers, likely due to their frequent co-occurrence in similar contexts.

2.3 Word: *apple*

The word *apple* was not present in the vocabulary after preprocessing, and hence nearest neighbors could not be computed.
Observation: This highlights a limitation of vocabulary construction and filtering, where lower-frequency or domain-specific words may be dropped during preprocessing.

3 Task 3

List every feature included in your CRF and identify which ones were most important.

3.1 Introduction

This task focuses on performing Named Entity Recognition (NER) on the **CoNLL-2003 dataset**.We employ a **Conditional Random Field (CRF)** to model the joint probability of the entire label sequence given the input tokens. The main task here is to create hand crafted features.

3.2 Label Encoding and Tagset

The CoNLL-2003 dataset utilizes the IOB (Inside, Outside, Beginning) tagging scheme. To facilitate model training, the 9 distinct NER tags are mapped to integer indices as follows:

Table 6: NER Tag Mapping for CoNLL-2003

Tag	**Index**	**Tag**	**Index**
O (Outside)	0	I-LOC (Location)	6
B-PER (Person)	1	B-MISC (Miscellaneous)	7
I-PER (Person)	2	I-MISC (Miscellaneous)	8
B-ORG (Organization)	3		
I-ORG (Organization)	4		
B-LOC (Location)	5		

3.3 Feature Engineering Methodology

A robust feature set was developed to provide the CRF with local and contextual information:

- **Lexical & Contextual Window:** For each token at index i, we extract the lowercased word and its neighboring words within a window of $[-2, +2]$. This allows the model to capture local dependencies, such as "New" often preceding "York".
- **Syntactic Features:** Part-of-Speech (POS) tags are included for the current and adjacent words to provide grammatical grounding.
- **Word Shape & Morphology:**
  - **Shape Mapping:** A transformation function maps characters to simplified patterns (e.g., "Apple" $\rightarrow$ "Xxxxx") to help the model generalize to unseen entities.
  - **Character N-Grams:** Prefixes and suffixes of lengths 2 to 5 are extracted to identify common entity-bearing morphemes like "-inc" or "-stan".
  - **Boolean Flags:** Binary features indicate capitalization patterns (`is_all_caps`, `is_istitle`) and the presence of digits.
- **Sequence Markers:** `BOS` (Beginning of Sentence) and `EOS` (End of Sentence) flags are applied to handle boundary conditions.

3.4 Model Configuration

The model is implemented using the `sklearn-crfsuite` library.

- **Algorithm:** L-BFGS (Limited-memory Broyden–Fletcher–Goldfarb–Shanno).
- **Regularization:** Elastic Net regularization is applied with L_1 ($c_1 = 0.1$) and L_2 ($c_2 = 0.1$) coefficients to ensure sparsity and prevent over-fitting.
- **State Transitions:** The model is configured to learn all possible transitions to build a more resilient transition matrix.

3.5 Results and Analysis

The model was evaluated on the CoNLL-2003 test set. The evaluation metrics focus on the 8 active entity tags, excluding the "O" (Outside) tag to provide a realistic assessment of entity detection. **Interpretation:**

Table 7 shows the performance of the optimized CRF model on the CoNLL-2003 test set. The model achieves an overall weighted F1-score of 0.811, indicating strong performance using hand-crafted features. Person and location entities obtain the highest F1-scores, showing that the model effectively captures capitalization and contextual patterns for these categories. Organization and miscellaneous entities, especially multi-token ones, show relatively lower performance due to more complex and varied structures. Overall, the CRF model demonstrates balanced and effective performance across most entity types.

Entity Tag	**Precision**	**Recall**	**F1-score**	**Support**
B-PER (1)	0.867	0.888	0.877	1617
I-PER (2)	0.900	0.968	0.933	1156
B-ORG (3)	0.823	0.779	0.800	1661
I-ORG (4)	0.701	0.792	0.744	835
B-LOC (5)	0.862	0.882	0.872	1668
I-LOC (6)	0.746	0.720	0.733	257
B-MISC (7)	0.840	0.793	0.816	702
I-MISC (8)	0.689	0.699	0.694	216
Micro Avg	0.833	0.848	0.840	8112
Macro Avg	0.804	0.815	0.809	8112
Weighted Avg	0.834	0.848	0.840	8112

Table 7: Classification report for the optimized CRF model on the CoNLL-2003 test set (excluding 'O' tag).

Feature Name	Weight
('is_first', '4')	-6.0397
('+2:word.lower():yellow', '5')	5.8722
('suffix-3:day', '0')	5.7827
('+1:word.shape:dddd-dd-dd', '5')	5.1992
('-1:word.lower():v', '3')	5.1702
('is_first', '2')	-4.9581
('word_bigram_next:3_arose', '3')	4.6345
('is_first', '8')	-4.2946
('+1:word.lower():arose', '3')	4.2366
('+1:word.shape:ddxx', '1')	4.1133
('+1:word.lower():arose', '0')	-4.1044
('postag:3', '7')	4.0939
('-1:word.lower():colo', '6')	4.0662
('word_bigram_prev:colo_.', '6')	4.0662
('+1:word.shape:dddd-dd-dd', '6')	4.0142
('word.lower():german', '7')	3.9552
('-1:word.lower():wisc', '6')	3.9291
('word_bigram_prev:wisc_.', '6')	3.9291
('is_all_lower', '0')	3.9268
('-1:word.shape:d.', '1')	3.9032

Table 8: Top 20 weighted features learned by the CRF model.

3.5.1 Top Feature Indicators

Table 8 presents the top features with the highest absolute weights learned by the CRF model. These weights indicate the relative importance of each feature in predicting named entity labels. Features with large positive weights strongly increase the likelihood of a particular label, while large negative weights discourage the corresponding label.

From the table, contextual features such as previous and next words (e.g., `-1:word.lower():wisc`, `+1:word.lower():arose`) and bigram combinations have high positive weights, indicating that surrounding context plays a crucial role in determining entity types. Word shape features, particularly date patterns such as `dddd-dd-dd`, strongly influence location predictions, reflecting structural patterns commonly found in news text.

Suffix-based features like `suffix-3:day` have high positive weights for the non-entity label, helping the model correctly classify tokens such as weekdays as outside named entities. Positional features such as `is_first` have strong negative weights for inside-entity labels, ensuring valid BIO tagging by discouraging incorrect entity continuation at the start of sentences. Additionally, lexical features and capitalization patterns further contribute to accurate classification by capturing common naming conventions.

Overall, the table demonstrates that contextual lexical features, word shape patterns, and positional constraints were the most influential in the CRF model's predictions, highlighting the importance of feature engineering in traditional sequence labeling approaches.

3.6 Comparative Analysis of Feature Engineering and Hyperparameter Tuning

Two versions of the CRF model were developed using different feature combinations and optimization strategies. The initial feature set consisted of basic lexical features (word identity, lowercase form), capitalization indicators, prefix–suffix information, and limited contextual features (± 1 word window). The improved feature set incorporated richer linguistic and contextual information, including word-shape features, extended context windows (± 2), bigram context features, conjunction-based patterns, and additional structural cues such as positional indicators and numeric/date patterns. These enhancements enabled the model to capture more informative dependencies between tokens and improved generalization for unseen entities.

In addition to feature improvements, hyperparameter tuning using randomized search was applied to optimize the regularization parameters of the CRF model. This further enhanced model performance by improving generalization and reducing overfitting.

Table 9 shows a comparison of performance across three configurations: the basic feature set, the enhanced feature set without hyperparameter tuning, and the final optimized model with hyperparameter tuning.

Model Configuration	**Token Accuracy**	**Entity-only Accuracy**	**Macro F1**
Basic feature set (initial model)	0.9594	0.8166	0.787
Enhanced feature set (no tuning)	0.9627	0.8436	0.805
Enhanced feature set + hyperparameter tuning (final model)	**0.9640**	**0.8490**	**0.811**

Table 9: Performance comparison of different CRF feature configurations and optimization strategies.

From the table, it is evident that incorporating richer contextual and structural features significantly improved entity-level accuracy from 0.8166 to 0.8436. This demonstrates that extended context windows, word-shape features, and bigram patterns help the model better capture dependencies and entity boundaries. Further improvements were achieved through hyperparameter tuning, increasing entity-only accuracy to 0.8490 and token-level accuracy to 0.9640.

3.7 Conclusion

The experimental results demonstrate that richer contextual and structural feature engineering significantly improves CRF performance for named entity recognition. The enhanced feature set improved entity-level accuracy and macro

F1-score over the basic feature set, while hyperparameter tuning provided further refinement and stability. The final optimized model achieved the best overall performance, highlighting the importance of comprehensive feature design and parameter optimization in traditional CRF-based NER systems.

4 Task 4

4.0.1 MLP Architecture

We implement a token-level Multi-Layer Perceptron (MLP) classifier for Named Entity Recognition (NER). Each input token is represented using a fixed pretrained embedding obtained either from:

- GloVe embeddings trained in Task 1, or
- Raw truncated Singular Value Decomposition (SVD) embeddings from Task 2.

Out-of-vocabulary (OOV) tokens are mapped to zero vectors.
The MLP consists of two fully connected layers:

- **Input Layer:** Word embedding of dimension d.
- **Hidden Layer:** Fully connected layer with 512 neurons and ReLU activation.
- **Output Layer:** Linear layer projecting to the number of NER classes, followed by softmax.

Formally, the model computes:

$$h = ReLU(W_1 x + b_1), \quad y = W_2 h + b_2$$

where $x \in R^d$ is the embedding vector for the input token.

Training is performed using the AdamW optimizer with learning rate 3×10^{-3} for 25 epochs. Cross-entropy loss is used while ignoring padded tokens. Evaluation is reported using Token-level Accuracy and Macro-F1 score on the CoNLL-2003 test set.

4.0.2 Quantitative Results

Table 10 presents the Token Accuracy and Macro-F1 scores for GloVe-MLP and Raw SVD-MLP across embedding dimensions $d \in \{50, 100, 200, 300\}$.

Model	Dim	Accuracy (%)	Macro-F1 (%)
GloVe-MLP	50	89.54	56.68
GloVe-MLP	100	90.10	59.56
GloVe-MLP	200	90.25	58.98
GloVe-MLP	300	**90.43**	**61.42**
Raw SVD-MLP	50	82.96	21.64
Raw SVD-MLP	100	83.05	24.09
Raw SVD-MLP	200	82.90	23.60
Raw SVD-MLP	300	83.01	21.54

Table 10: Token Accuracy and Macro-F1 scores for GloVe-MLP and Raw SVD-MLP models.

4.0.3 Best Configuration

The best overall performance is obtained using:

- **Model:** GloVe-MLP
- **Embedding Dimension:** $d = 300$
- **Macro-F1 Score:** 61.42%

This configuration achieves the highest balanced performance across all entity classes.

4.1 Label-wise Performance Comparison: Best SVD vs Best GloVe

We compare the per-label F1-scores obtained using the best-performing SVD embeddings and the best-performing GloVe embeddings. This fine-grained evaluation highlights how embedding quality impacts token-level classification across different named entity categories.

Observation: GloVe embeddings substantially outperform SVD embeddings across all named entity classes, with especially large gains for `PER`, `ORG`, and `LOC` categories. While SVD embeddings achieve reasonable performance on the dominant `O` class, they fail to capture the fine-grained semantic and syntactic distinctions required for entity recognition. In contrast, GloVe's predictive training objective and global co-occurrence modeling yield significantly richer representations, enabling robust discrimination between entity types. Overall, this demonstrates that distributional predictive embeddings are far better suited than low-rank matrix factorization for downstream structured prediction tasks like NER.

Label	Best SVD F1	Best GloVe F1
O	0.9076	0.9652
B-PER	0.0147	0.5339
I-PER	0.0234	0.3118
B-ORG	0.0337	0.5881
I-ORG	0.1323	0.5148
B-LOC	0.1352	0.7593
I-LOC	0.1565	0.5595
B-MISC	0.2445	0.6846
I-MISC	0.5202	0.6108

Table 11: Label-wise F1-score comparison between best SVD-based embeddings and best GloVe embeddings.

4.1.1 Analysis of SVD Performance

Raw SVD-based embeddings perform significantly worse than GloVe embeddings, achieving Macro-F1 scores in the range of only 21–24%. Although token-level accuracy remains moderately high (around 83%), this is misleading due to severe class imbalance in the dataset, where the majority class is the non-entity label `O`.

Failure Analysis: Raw SVD embeddings are derived from low-rank factorization of the term-document matrix, which primarily captures global word frequency and document co-occurrence patterns. This representation lacks:

- Fine-grained syntactic structure,
- Local contextual semantics,
- Sequential dependency modeling.

As a result, the classifier heavily over-predicts the majority class `O`, leading to extremely low recall for entity labels such as PER, ORG, and LOC. This behavior is clearly reflected in the near-zero recall values for several named entity classes.

In contrast, GloVe embeddings are trained using local context windows and directly optimize for semantic similarity, yielding significantly richer lexical representations.

4.1.2 Comparison with CRF

We further compare the best-performing MLP model against a Conditional Random Field (CRF) baseline.

CRF consistently outperforms both GloVe-MLP and Raw SVD-MLP models. This performance gain arises from CRF's ability to model structured dependencies between output labels, enforcing valid tag transitions and capturing strong sequential correlations inherent in NER tasks.

Discussion: While MLP models treat each token independently, CRF models explicitly learn transition probabilities between tags, enabling them to enforce global consistency constraints (e.g., `I-ORG` cannot follow `B-PER`). This structural modeling significantly improves boundary detection and entity coherence.

Although CRFs achieve superior accuracy, MLP models benefit from simpler architectures, faster training, and ease of implementation. Combining neural embeddings with structured decoding, such as BiLSTM-CRF or Transformer-CRF, can further bridge this performance gap.

4.1.3 Conclusion

Our experiments demonstrate that:

- GloVe embeddings substantially outperform raw SVD embeddings for sequence labeling tasks.
- Pure matrix factorization methods fail to capture the syntactic and semantic complexity required for NER.
- Structured prediction models like CRF remain superior due to explicit modeling of label dependencies.

Overall, the combination of distributional word embeddings with structured decoding emerges as the most effective approach for NER.

5 Task 5: SVD Performance Boost using TF-IDF

In this task, we investigate whether applying a TF-IDF weighting scheme to the term-document matrix prior to Singular Value Decomposition (SVD) can improve embedding quality and downstream performance. We compare TF-IDF weighted SVD embeddings against the Raw SVD embeddings obtained in Task 4 using two quality checks.

5.0.1 TF-IDF Weighted SVD

Instead of directly factorizing the raw term-document count matrix X, we first apply TF-IDF transformation:

$$X_{tfidf} = TF - IDF(X)$$

We then perform truncated SVD on X_{tfidf}:

$$X_{tfidf} \approx U_k \Sigma_k V_k^\top$$

The final word embeddings are computed as:

$$E = U_k \Sigma_k$$

where k denotes the embedding dimension. We use the best-performing SVD dimension from Task 4, namely $d = 100$.

5.0.2 Quality Check 1: Nearest Neighbor Analysis

We select five diverse probe words: `king` (noun), `computer` (technology), `play` (verb), `good` (adjective), and `year` (temporal noun). For each word, we compare the top-5 nearest neighbors obtained using Raw SVD embeddings versus TF-IDF SVD embeddings.

Word	**Raw SVD Neighbors**	**TF-IDF SVD Neighbors**
king	Greatest, Christopher, nations, edged, creates	palace, Palace, missionaries, rousing, Protestant
computer	priced, Front, ill-fated, Let, Sport	Let, aluminum, ancestor, Apple, discontinued
play	American, experiences, If, traditional, past	played, career, 11-game, foe, day
good	race, service, work, help, required	work, did, should, see, while
year	3, December, want, worth, like	New, million, technology, start, platform

Table 12: Top-5 nearest neighbors comparison: Raw SVD vs TF-IDF SVD.

Observations: Across all probe words, TF-IDF weighted SVD embeddings yield significantly more semantically meaningful neighbors compared to Raw SVD. Raw SVD neighbors often appear unrelated or noisy, reflecting dominance of high-frequency but uninformative terms.

TF-IDF weighting suppresses frequent function words and amplifies discriminative terms, resulting in improved semantic coherence. For example:

- `king` shifts from unrelated words toward *palace* and religious or historical terms.
- `computer` gains meaningful neighbors such as *Apple*.
- `play` retrieves syntactically and semantically relevant forms such as *played*.

This demonstrates that TF-IDF weighting significantly enhances the semantic quality of SVD embeddings.

5.0.3 Quality Check 2: Downstream NER Performance

We train a final MLP classifier using TF-IDF SVD embeddings and evaluate it on the CoNLL-2003 NER test set.

Observations: Despite noticeable improvements in nearest-neighbor quality, TF-IDF weighted SVD does not improve downstream NER performance. In fact, a slight degradation in Macro-F1 score is observed compared to Raw SVD.

Embedding Type	Dim	Accuracy (%)	Macro-F1 (%)
Raw SVD-MLP	100	83.05	24.09
TF-IDF SVD-MLP	100	82.75	20.04

Table 13: Comparison of Raw SVD vs TF-IDF SVD for NER.

5.0.4 Discussion

The contradictory behavior between qualitative and quantitative evaluation highlights a fundamental limitation of count-based matrix factorization methods for sequence labeling tasks.

TF-IDF improves semantic similarity by emphasizing discriminative lexical features, which benefits nearest-neighbor retrieval. However, Named Entity Recognition relies heavily on:

- Local contextual patterns,
- Sequential label dependencies,
- Morphosyntactic cues.

SVD embeddings—whether raw or TF-IDF weighted—remain fundamentally context-independent. As a result, although TF-IDF improves word similarity, it fails to encode the structural information required for accurate sequence labeling.

5.0.5 Conclusion

This experiment demonstrates that:

- TF-IDF weighting significantly improves the semantic coherence of SVD embeddings.
- However, this improvement does not translate into better downstream NER performance.
- Pure matrix factorization methods are fundamentally limited for structured prediction tasks such as NER.

These findings further motivate the use of neural embedding models and structured prediction frameworks for sequence labeling tasks.

References

1. Medium Blog: Named Entity Recognition using Conditional Random Fields. `https://medium.com/data-science-in-your-pocket/named-entity-recognition-ner-using-conditional-random-fields-in-nlp-3660df22e95c`

2. Medium Blog: Training Custom NER System using CRFs. https://medium.com/data-science-in-your-pocket/training-custom-ner-system-using-crfs-146e0e922851

3. GitHub Repository: aleju/ner-crf. https://github.com/aleju/ner-crf

4. Gemini AI Discussion. https://gemini.google.com/share/b26162f3231e

5. ChatGPT Discussion. https://chatgpt.com/share/6990a578-ab7c-800c-ba44-e632544e2a3b